🚀 Day 21 – Sliding Window

🔥 Sliding Window kya hota hai?

👉 Ek technique hai jisme hum:

- Window (subarray / substring) maintain karte hain
- Expand aur shrink karte hain instead of brute force

👉 Brute Force = $O(n^2)$
👉 Sliding Window = $O(n)$ ✅

🧠 Types of Sliding Window

1️⃣ Fixed Size Window

👉 Window ka size fix hota hai (k)

2️⃣ Variable Size Window

👉 Condition ke basis pe window expand/shrink hoti hai

🚀 1. Maximum Sum Subarray of Size K (Fixed Window)

📌 Problem

Size k ka subarray jiska sum maximum ho.

💡 Approach

- First k elements ka sum lo
- Phir window slide karo

✅ Code

```
public class MaxSumSubarray {
   public static int maxSum(int[] arr, int k){
      int windowSum = 0;

      // First window
      for(int i = 0; i < k; i++){
         windowSum += arr[i];
      }

      int maxSum = windowSum;

      for(int i = k; i < arr.length; i++){
         windowSum += arr[i];      // add next
         windowSum -= arr[i - k];  // remove previous

         maxSum = Math.max(maxSum, windowSum);
      }

      return maxSum;
   }
}
```

⏱️ Complexity

- O(n)

🚀 2. First Negative Number in Every Window (Fixed)

📌 Problem

Har window size k me first negative number find karo.

💡 Approach

- Queue use karo negative numbers store karne ke liye

✅ **Code**

```
import java.util.*;

public class FirstNegative {
    public static void firstNegative(int[] arr, int k){
        Queue<Integer> q = new LinkedList<>();

        for(int i = 0; i < arr.length; i++){
            if(arr[i] < 0){
                q.add(i);
            }

            // window size reach
            if(i >= k - 1){
                // remove out of window
                while(!q.isEmpty() && q.peek() <= i - k){
                    q.poll();
                }

                if(q.isEmpty()){
                    System.out.print("0 ");
                } else {
                    System.out.print(arr[q.peek()] + " ");
                }
            }
        }
    }
}
```

🚀 **3. Longest Substring Without Repeating Characters** 🔥

(Revision – Important)

💡 **Approach**

- HashSet + two pointers

✅ Code

```java
import java.util.*;

public class LongestSubstring {
    public static int lengthOfLongestSubstring(String s){
        HashSet<Character> set = new HashSet<>();

        int left = 0;
        int maxLength = 0;

        for(int right = 0; right < s.length(); right++){
            while(set.contains(s.charAt(right))){
                set.remove(s.charAt(left));
                left++;
            }

            set.add(s.charAt(right));
            maxLength = Math.max(maxLength, right - left + 1);
        }

        return maxLength;
    }
}
```

🚀 4. Longest Subarray with Sum ≤ K (Variable Window)

📌 Problem

Longest subarray jiska sum ≤ k ho

💡 Approach

- Expand window
- Sum > k → shrink window

✅ Code

```
public class LongestSubarray {
    public static int longestSubarray(int[] arr, int k){
        int left = 0, sum = 0, maxLen = 0;

        for(int right = 0; right < arr.length; right++){
            sum += arr[right];

            while(sum > k){
                sum -= arr[left];
                left++;
            }

            maxLen = Math.max(maxLen, right - left + 1);
        }

        return maxLen;
    }
}
```

🚀 5. Minimum Window Substring 🔥 🔥 (Advanced)

📌 Problem

String s me smallest substring find karo jo t ke sab characters contain kare

💡 Approach

- Frequency map
- Match count maintain karo
- Window shrink karo minimum ke liye

✅ Code

```java
import java.util.*;

public class MinWindow {
    public static String minWindow(String s, String t){
        HashMap<Character, Integer> map = new HashMap<>();

        for(char ch : t.toCharArray()){
            map.put(ch, map.getOrDefault(ch, 0) + 1);
        }

        int left = 0, count = 0, minLen = Integer.MAX_VALUE;
        int start = 0;

        for(int right = 0; right < s.length(); right++){
            char ch = s.charAt(right);

            if(map.containsKey(ch)){
                map.put(ch, map.get(ch) - 1);
                if(map.get(ch) >= 0){
                    count++;
                }
            }

            while(count == t.length()){
                if(right - left + 1 < minLen){
                    minLen = right - left + 1;
                    start = left;
                }

                char leftChar = s.charAt(left);

                if(map.containsKey(leftChar)){
                    map.put(leftChar, map.get(leftChar) + 1);
                    if(map.get(leftChar) > 0){
                        count--;
                    }
                }

                left++;
            }
        }

        return minLen == Integer.MAX_VALUE ? "" : s.substring(start, start + minLen);
    }
}
```

🎯 Sliding Window Pattern (IMPORTANT 🔥)

🧠 Fixed Window

1. First window process karo
2. Slide karo (add next, remove previous)

🧠 Variable Window

1. Right pointer expand
2. Condition check
3. Left pointer shrink
4. Answer update